

CS620T: Compilers - Principles and Implementation

Autumn 2026

Week 3: Design and Implementation of
scanners and parsers contd..

Parsers – what do we need to know?

1. How do we define language constructs?
 - Context-free grammars
2. How do we write Parsers?
 - E.g. use a parser generator tool such as [Bison](#)
3. How do we determine: 1) valid strings in the language? 2) structure of program?
 - [LL Parsers, LR Parsers](#)



Next

Top-down Parsing

- Idea: we know sentence has to start with initial symbol
- Build up partial derivations by *predicting* what rules are used to expand non-terminals
- Often called *predictive parsers*
- If partial derivation has terminal characters, *match* them from the input stream

Top-down Parsing

- Also called recursive-descent parsing
- Equivalent to finding the left-derivation for an input string
 - Recall: expand the leftmost non-terminal in a parse tree
 - Expand the parse tree in pre-order i.e., identify parent nodes before children

Top-down Parsing

↑: next symbol to
be read

1: $S \rightarrow cAd$
2: $A \rightarrow ab$
3: | a

Step	Input string	Parse tree
1	cad ↑	S

String: cad

Start with S

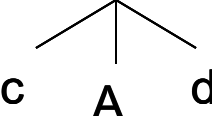
Top-down Parsing

↑: next symbol to be read

1: $S \rightarrow cAd$

2: $A \rightarrow ab$

3: | a

Step	Input string	Parse tree
1	cad	S
2	↑ cad ↑	

String: cad

Predict rule 1

Top-down Parsing

↑: next symbol to be read

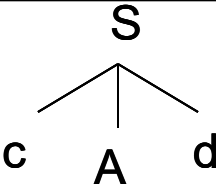
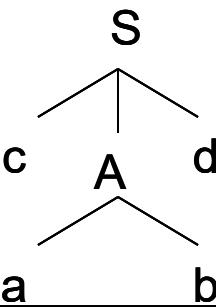
1: $S \rightarrow cAd$

2: $A \rightarrow ab$

3: | a

String: cad

Predict rule 2

Step	Input string	Parse tree
1	cad	S
2	↑ cad ↑	
3	cad ↑	

Top-down Parsing

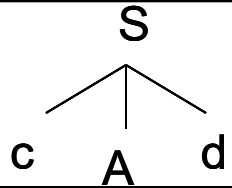
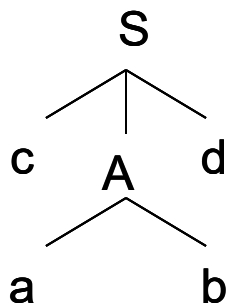
↑: next symbol to be read

1: $S \rightarrow cAd$

2: $A \rightarrow ab$

3: | a

String: cad

Step	Input string	Parse tree
1	cad ↑	S
2	cad ↑	 <pre>graph TD; S1[S] --- c1[c]; S1 --- A1[A]; S1 --- d1[d];</pre>
3	cad ↑	 <pre>graph TD; S2[S] --- c2[c]; S2 --- A2[A]; S2 --- d2[d]; A2 --- a[a]; A2 --- b[b];</pre>

No more non terminals!

String doesn't match.

Backtrack.

Top-down Parsing

↑: next symbol to be read

1: $S \rightarrow cAd$

2: $A \rightarrow ab$

3: | a

Step	Input string	Parse tree
1	cad	S
2	↑ cad ↑	<pre>graph TD; S --> c; S --> A; S --> d; A --> a; A --> b; A --> a;</pre>

String: cad

Top-down Parsing

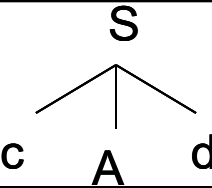
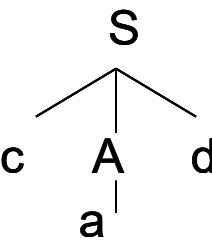
↑: next symbol to be read

1: $S \rightarrow cAd$

2: $A \rightarrow ab$

3: | a

String: cad

Step	Input string	Parse tree
1	cad	S
2	↑ cad ↑	
4	cad ↑	

Predict rule 3

Top-down Parsing – Table-driven Approach

1: $S \rightarrow F$

2: $S \rightarrow (S + F)$

3: $F \rightarrow a$

string: (a+a)

string': (a+a)\$

	(	)	a	+	\$
S	2	-	1	-	-
F	-	-	3	-	-

Assume that the table is given.

Top-down Parsing – Table-driven Approach

1: $S \rightarrow F$

2: $S \rightarrow (S + F)$

3: $F \rightarrow a$

string: (a+a)

string': (a+a)\$

	(	)	a	+	\$
S	2	-	1	-	-
F	-	-	3	-	-

Assume that the table is given.

- Table-driven (Parse Table) approach doesn't require backtracking

But how do we construct such a table?

Important Concepts: First Sets and Follow Sets

First and follow sets

- $\text{First}(\alpha)$: the set of terminals (and/or λ) that begin all strings that can be derived from α

- $\text{First}(A) = \{x, y, \lambda\}$

- $\text{First}(xaA) = \{x\}$

- $\text{First}(AB) = \{x, y, b\}$

- $\text{Follow}(A)$: the set of terminals (and/or \$, but no λ s) that can appear immediately after A in some partial derivation

- $\text{Follow}(A) = \{b\}$

$$S \rightarrow A B \$$$

$$A \rightarrow x a A$$

$$A \rightarrow y a A$$

$$A \rightarrow \lambda$$

$$B \rightarrow b$$

First and follow sets

- $\text{First}(\alpha) = \{a \in V_t \mid \alpha \Rightarrow^* a\beta\} \cup \{\lambda \mid \text{if } \alpha \Rightarrow^* \lambda\}$
- $\text{Follow}(A) = \{a \in V_t \mid S \Rightarrow^+ \dots Aa \dots\} \cup \{\$ \mid \text{if } S \Rightarrow^+ \dots A \$\}$

S: start symbol

a: a terminal symbol

A: a non-terminal symbol

α, β : a string composed of terminals and non-terminals (typically, α is the RHS of a production

$\Rightarrow$: derived in 1 step

$\Rightarrow^*$: derived in 0 or more steps

$\Rightarrow^+$: derived in 1 or more steps

Computing first sets

- Terminal: $\text{First}(a) = \{a\}$
- Non-terminal: $\text{First}(A)$
 - Look at all productions for A
$$A \rightarrow X_1 X_2 \dots X_k$$
 - $\text{First}(A) \supseteq (\text{First}(X_1) - \lambda)$
 - If $\lambda \in \text{First}(X_1)$, $\text{First}(A) \supseteq (\text{First}(X_2) - \lambda)$
 - If λ is in $\text{First}(X_i)$ for all i , then $\lambda \in \text{First}(A)$
- Computing $\text{First}(\alpha)$: similar procedure to computing $\text{First}(A)$

Towards parser generators

- Key problem: as we read the source program, we need to decide what productions to use
- Step 1: find the tokens that can tell which production P (of the form $A \rightarrow X_1 X_2 \dots X_m$) applies

$\text{Predict}(P) =$

$$\begin{cases} \text{First}(X_1 \dots X_m) & \text{if } \lambda \notin \text{First}(X_1 \dots X_m) \\ (\text{First}(X_1 \dots X_m) - \lambda) \cup \text{Follow}(A) & \text{otherwise} \end{cases}$$

- If next token is in $\text{Predict}(P)$, then we should choose this production

First Set - Example

- 1) $S \rightarrow ABc\$$
- 2) $A \rightarrow xaA$
- 3) $A \rightarrow yaA$
- 4) $A \rightarrow c$
- 5) $B \rightarrow b$
- 6) $B \rightarrow \lambda$

$\text{first}(S) = \{ \text{?} \}$

Think of all possible strings derivable from S .
Get the **first terminal symbol** in those strings
or λ if S derives λ

First Set - Example

- 1) $S \rightarrow ABc\$$
- 2) $A \rightarrow xaA$
- 3) $A \rightarrow yaA$
- 4) $A \rightarrow c$
- 5) $B \rightarrow b$
- 6) $B \rightarrow \lambda$

$\text{first}(S) = \{ x, y, c \}$

First Set - Example

- 1) $S \rightarrow ABc\$$
- 2) $A \rightarrow xaA$
- 3) $A \rightarrow yaA$
- 4) $A \rightarrow c$
- 5) $B \rightarrow b$
- 6) $B \rightarrow \lambda$

$\text{first}(S) = \{ x, y, c \}$

$\text{first}(A) = \{ ? \}$

First Set - Example

- 1) $S \rightarrow ABc\$$
- 2) $A \rightarrow xaA$
- 3) $A \rightarrow yaA$
- 4) $A \rightarrow c$
- 5) $B \rightarrow b$
- 6) $B \rightarrow \lambda$

$\text{first}(S) = \{ x, y, c \}$

$\text{first}(A) = \{ x, y, c \}$

First Set - Example

- 1) $S \rightarrow ABc\$$
- 2) $A \rightarrow xaA$
- 3) $A \rightarrow yaA$
- 4) $A \rightarrow c$
- 5) $B \rightarrow b$
- 6) $B \rightarrow \lambda$

$\text{first}(S) = \{ x, y, c \}$
 $\text{first}(A) = \{ x, y, c \}$
 $\text{first}(B) = \{ ? \}$

First Set - Example

- 1) $S \rightarrow ABc\$$
- 2) $A \rightarrow xaA$
- 3) $A \rightarrow yaA$
- 4) $A \rightarrow c$
- 5) $B \rightarrow b$
- 6) $B \rightarrow \lambda$

$\text{first}(S) = \{ x, y, c \}$
 $\text{first}(A) = \{ x, y, c \}$
 $\text{first}(B) = \{ b, \lambda \}$

Follow Set - Example

- 1) $S \rightarrow ABc\$$
- 2) $A \rightarrow xaA$
- 3) $A \rightarrow yaA$
- 4) $A \rightarrow c$
- 5) $B \rightarrow b$
- 6) $B \rightarrow \lambda$

$\text{follow}(S) = \{ \text{?} \}$

Think of all strings **possible in the language** having the form $..Sa..$. Get the **following terminal symbol** a after S in those strings or $\$$ if you get a string $..S\$$

Follow Set - Example

- 1) $S \rightarrow ABc\$$
- 2) $A \rightarrow xaA$
- 3) $A \rightarrow yaA$
- 4) $A \rightarrow c$
- 5) $B \rightarrow b$
- 6) $B \rightarrow \lambda$

$\text{follow}(S) = \{ \quad \}$

Follow Set - Example

- 1) $S \rightarrow ABc\$$
- 2) $A \rightarrow xaA$
- 3) $A \rightarrow yaA$
- 4) $A \rightarrow c$
- 5) $B \rightarrow b$
- 6) $B \rightarrow \lambda$

$\text{follow}(S) = \{ \quad \}$
 $\text{follow}(A) = \{ \text{?} \}$

Follow Set - Example

- 1) $S \rightarrow ABc\$$
- 2) $A \rightarrow xaA$
- 3) $A \rightarrow yaA$
- 4) $A \rightarrow c$
- 5) $B \rightarrow b$
- 6) $B \rightarrow \lambda$

$\text{follow}(S) = \{ \quad \}$

$\text{follow}(A) = \{ b, c \}$

e.g. $xaAbc\$$, $xaAc\$$

Follow Set - Example

- 1) $S \rightarrow ABc\$$
- 2) $A \rightarrow xaA$
- 3) $A \rightarrow yaA$
- 4) $A \rightarrow c$
- 5) $B \rightarrow b$
- 6) $B \rightarrow \lambda$

$\text{follow}(S) = \{ \quad \}$

$\text{follow}(A) = \{ b, c \}$ e.g. $xaAbc\$$, $xaAc\$$

What happens when you consider: $A \rightarrow xaA$ or $A \rightarrow yaA$?

Follow Set - Example

- 1) $S \rightarrow ABc\$$
- 2) $A \rightarrow xaA$
- 3) $A \rightarrow yaA$
- 4) $A \rightarrow c$
- 5) $B \rightarrow b$
- 6) $B \rightarrow \lambda$

$\text{follow}(S) = \{ \quad \}$

$\text{follow}(A) = \{ b, c \}$ e.g. $xaAbc\$$, $xaAc\$$

What happens when you consider: $A \rightarrow xaA$ or $A \rightarrow yaA$?

- You will get string of the form $A \Rightarrow^+ (xa)^+A$
- But we need strings of the form: $\dots Aa \dots$ or $\dots Ab \dots$ or $\dots Ac \dots$

Follow Set - Example

- 1) $S \rightarrow ABc\$$
- 2) $A \rightarrow xaA$
- 3) $A \rightarrow yaA$
- 4) $A \rightarrow c$
- 5) $B \rightarrow b$
- 6) $B \rightarrow \lambda$

$\text{follow}(S) = \{ \quad \}$
 $\text{follow}(A) = \{ b, c \}$
 $\text{follow}(B) = \{ ? \}$

Follow Set - Example

- 1) $S \rightarrow ABc\$$
- 2) $A \rightarrow xaA$
- 3) $A \rightarrow yaA$
- 4) $A \rightarrow c$
- 5) $B \rightarrow b$
- 6) $B \rightarrow \lambda$

$\text{follow}(S) = \{ \quad \}$
 $\text{follow}(A) = \{ b, c \}$
 $\text{follow}(B) = \{ c \}$

Predict Set - Example

1) $S \rightarrow ABc\$$

2) $A \rightarrow xaA$

3) $A \rightarrow yaA$

4) $A \rightarrow c$

5) $B \rightarrow b$

6) $B \rightarrow \lambda$

$\text{Predict}(P) =$

$$\begin{cases} \text{First}(X_1 \dots X_m) & \text{if } \lambda \notin \text{First}(X_1 \dots X_m) \\ (\text{First}(X_1 \dots X_m) - \lambda) \cup \text{Follow}(A) & \text{otherwise} \end{cases}$$

$\text{Predict}(1) = \{ ? \} = \text{First}(ABc\$) \text{ if } \lambda \notin \text{First}(ABc\$)$

Predict Set - Example

- 1) $S \rightarrow ABc\$$
- 2) $A \rightarrow xaA$
- 3) $A \rightarrow yaA$
- 4) $A \rightarrow c$
- 5) $B \rightarrow b$
- 6) $B \rightarrow \lambda$

	x	y	a	b	c	\$
S	1	1			1	
A						
B						

Predict (1) = { x, y, c }

Predict Set - Example

- 1) $S \rightarrow ABc\$$
- 2) $A \rightarrow xaA$
- 3) $A \rightarrow yaA$
- 4) $A \rightarrow c$
- 5) $B \rightarrow b$
- 6) $B \rightarrow \lambda$

	x	y	a	b	c	\$
S	1	1			1	
A						
B						

Predict (1) = { x, y, c }

Predict (2) = { ? } = First(xaA) if $\lambda \notin \text{First}(xaA)$

Predict Set - Example

- 1) $S \rightarrow ABc\$$
- 2) $A \rightarrow xaA$
- 3) $A \rightarrow yaA$
- 4) $A \rightarrow c$
- 5) $B \rightarrow b$
- 6) $B \rightarrow \lambda$

	x	y	a	b	c	\$
S	1	1			1	
A	2					
B						

Predict (1) = { x, y, c }

Predict (2) = { x }

Predict Set - Example

- 1) $S \rightarrow ABc\$$
- 2) $A \rightarrow xaA$
- 3) $A \rightarrow yaA$
- 4) $A \rightarrow c$
- 5) $B \rightarrow b$
- 6) $B \rightarrow \lambda$

	x	y	a	b	c	\$
S	1	1			1	
A	2					
B						

Predict (1) = { x, y, c }

Predict (2) = { x }

Predict (3) = { ? } = First(yaA) if $\lambda \notin \text{First}(yaA)$

Predict Set - Example

- 1) $S \rightarrow ABc\$$
- 2) $A \rightarrow xaA$
- 3) $A \rightarrow yaA$
- 4) $A \rightarrow c$
- 5) $B \rightarrow b$
- 6) $B \rightarrow \lambda$

	x	y	a	b	c	\$
S	1	1			1	
A	2	3				
B						

Predict (1) = { x, y, c }

Predict (2) = { x }

Predict (3) = { y }

Predict Set - Example

- 1) $S \rightarrow ABc\$$
- 2) $A \rightarrow xaA$
- 3) $A \rightarrow yaA$
- 4) $A \rightarrow c$
- 5) $B \rightarrow b$
- 6) $B \rightarrow \lambda$

	x	y	a	b	c	\$
S	1	1			1	
A	2	3				
B						

Predict (1) = { x, y, c }

Predict (2) = { x }

Predict (3) = { y }

Predict (4) = { ? } = First(c) if $\lambda \notin \text{First}(c)$

Predict Set - Example

- 1) $S \rightarrow ABc\$$
- 2) $A \rightarrow xaA$
- 3) $A \rightarrow yaA$
- 4) $A \rightarrow c$
- 5) $B \rightarrow b$
- 6) $B \rightarrow \lambda$

	x	y	a	b	c	\$
S	1	1			1	
A	2	3			4	
B						

- Predict (1) = { x, y, c }
- Predict (2) = { x }
- Predict (3) = { y }
- Predict (4) = { c }

Predict Set - Example

- 1) $S \rightarrow ABc\$$
- 2) $A \rightarrow xaA$
- 3) $A \rightarrow yaA$
- 4) $A \rightarrow c$
- 5) $B \rightarrow b$
- 6) $B \rightarrow \lambda$

	x	y	a	b	c	\$
S	1	1			1	
A	2	3			4	
B						

Predict (1) = { x, y, c }

Predict (2) = { x }

Predict (3) = { y }

Predict (4) = { c }

Predict (5) = { ? } = First(b) if $\lambda \notin \text{First}(b)$

Predict Set - Example

- 1) $S \rightarrow ABc\$$
- 2) $A \rightarrow xaA$
- 3) $A \rightarrow yaA$
- 4) $A \rightarrow c$
- 5) $B \rightarrow b$
- 6) $B \rightarrow \lambda$

	x	y	a	b	c	\$
S	1	1			1	
A	2	3			4	
B				5		

Predict (1) = { x, y, c }
Predict (2) = { x }
Predict (3) = { y }
Predict (4) = { c }
Predict (5) = { b }

Predict Set - Example

- 1) $S \rightarrow ABc\$$
- 2) $A \rightarrow xaA$
- 3) $A \rightarrow yaA$
- 4) $A \rightarrow c$
- 5) $B \rightarrow b$
- 6) $B \rightarrow \lambda$

	x	y	a	b	c	\$
S	1	1			1	
A	2	3			4	
B				5		

Predict (1) = { x, y, c }

Predict (2) = { x }

Predict (3) = { y }

Predict (4) = { c }

Predict (5) = { b }

Predict (6) = { ? } = First(λ) ?

Predict(P) =

$$\begin{cases} \text{First}(X_1 \dots X_m) & \text{if } \lambda \notin \text{First}(X_1 \dots X_m) \\ (\text{First}(X_1 \dots X_m) - \lambda) \cup \text{Follow}(A) & \text{otherwise} \end{cases}$$

Predict Set - Example

- 1) $S \rightarrow ABc\$$
- 2) $A \rightarrow xaA$
- 3) $A \rightarrow yaA$
- 4) $A \rightarrow c$
- 5) $B \rightarrow b$
- 6) $B \rightarrow \lambda$

	x	y	a	b	c	\$
S	1	1			1	
A	2	3			4	
B				5		

Predict (1) = { x, y, c }

Predict (2) = { x }

Predict (3) = { y }

Predict (4) = { c }

Predict (5) = { b }

Predict (6) = { ? } = ~~First(λ)~~ ? Follow(B)

Predict(P) =

$$\begin{cases} \text{First}(X_1 \dots X_m) & \text{if } \lambda \notin \text{First}(X_1 \dots X_m) \\ (\text{First}(X_1 \dots X_m) - \lambda) \cup \text{Follow}(A) & \text{otherwise} \end{cases}$$

Predict Set - Example

- 1) $S \rightarrow ABc\$$
- 2) $A \rightarrow xaA$
- 3) $A \rightarrow yaA$
- 4) $A \rightarrow c$
- 5) $B \rightarrow b$
- 6) $B \rightarrow \lambda$

	x	y	a	b	c	\$
S	1	1			1	
A	2	3			4	
B				5	6	

Predict (1) = { x, y, c }

Predict (2) = { x }

Predict (3) = { y }

Predict (4) = { c }

Predict (5) = { b }

Predict (6) = { λ }

Computing Parse-Table

- 1) $S \rightarrow ABc\$$
- 2) $A \rightarrow xaA$
- 3) $A \rightarrow yaA$
- 4) $A \rightarrow c$
- 5) $B \rightarrow b$
- 6) $B \rightarrow \lambda$

	x	y	a	b	c	\$
S	1	1			1	
A	2	3			4	
B				5	6	

$\text{first}(S) = \{x, y, c\}$
 $\text{first}(A) = \{x, y, c\}$
 $\text{first}(B) = \{b, \lambda\}$

$\text{follow}(S) = \{\}$
 $\text{follow}(A) = \{b, c\}$
 $\text{follow}(B) = \{c\}$

$P(1) = \{x, y, c\}$
 $P(2) = \{x\}$
 $P(3) = \{y\}$
 $P(4) = \{c\}$
 $P(5) = \{b\}$
 $P(6) = \{c\}$

Top-Down Parsing - Example

string: xacc\$

Stack

?

Rem. Input

xacc\$

Action

?

What do you put on the stack?

Top-Down Parsing - Example

string: xacc\$

Stack

?

Rem. Input

xacc\$

Action

?

What do you put on the stack? – strings that you derive

Top-Down Parsing - Example

string: xacc\$

Stack*

S

Rem. Input

xacc\$

Action

?

Top-down parsing. So, start with S.

Top-Down Parsing - Example

string: xacc\$

Stack*	Rem. Input	Action
S	xacc\$	?

Top-down parsing. So, start with S.

What action do you take when stack-top has symbol S and the string to be matched has terminal x in front?

Top-Down Parsing - Example

string: xacc\$

Stack*

S

Rem. Input

xacc\$

Action

Predict(1) S → ABc\$

Top-down parsing. So, start with S.

What action do you take when stack-top has **symbol S** and the string to be matched has **terminal x** in front? – consult parse table

	x	y	a	b	c	\$
S	1	1			1	
A	2	3			4	
B				5	6	

Top-Down Parsing - Example

string: xacc\$

Stack*

S
ABc\$

Rem. Input

xacc\$

Action

Predict(1) S → ABc\$

	x	y	a	b	c	\$
S	1	1			1	
A	2	3			4	
B				5	6	

Top-Down Parsing - Example

string: xacc\$

Stack*

S
ABc\$

Rem. Input

xacc\$
xacc\$

Action

Predict(1) S->ABc\$

*What action do you take when stack-top has **symbol A** and the string to be matched has **terminal x** in front? – consult parse table*

	x	y	a	b	c	\$
S	1	1			1	
A	2	3			4	
B				5	6	

Top-Down Parsing - Example

string: xacc\$

Stack*	Rem. Input	Action
S	xacc\$	Predict(1) S → ABc\$
ABc\$	xacc\$	Predict(2) A → xaA

What action do you take when stack-top has **symbol A** and the string to be matched has **terminal x** in front? – consult parse table

	x	y	a	b	c	\$
S	1	1			1	
A	2	3			4	
B				5	6	

Top-Down Parsing - Example

string: xacc\$

Stack*

S

ABc\$

xaABc\$

Rem. Input

xacc\$

xacc\$

Action

Predict(1) S → ABc\$

Predict(2) A → xaA

	x	y	a	b	c	\$
S	1	1			1	
A	2	3			4	
B				5	6	

Top-Down Parsing - Example

string: xacc\$

Stack*

S

ABc\$

xaABc\$

Rem. Input

xacc\$

xacc\$

xacc\$

Action

Predict(1) S → ABc\$

Predict(2) A → xaA

?

	x	y	a	b	c	\$
S	1	1			1	
A	2	3			4	
B				5	6	

Top-Down Parsing - Example

string: xacc\$

Stack*	Rem. Input	Action
S	xacc\$	Predict(1) S->ABc\$
ABc\$	xacc\$	Predict(2) A->xaA
x aABc\$	x acc\$	match(x)

	x	y	a	b	c	\$
S	1	1			1	
A	2	3			4	
B				5	6	

Top-Down Parsing - Example

string: xacc\$

Stack*	Rem. Input	Action
S	xacc\$	Predict(1) S->ABc\$
ABc\$	xacc\$	Predict(2) A->xaA
xaABc\$	xacc\$	match(x)
aABc\$	acc\$	match(a)

	x	y	a	b	c	\$
S	1	1			1	
A	2	3			4	
B				5	6	

Top-Down Parsing - Example

string: xacc\$

Stack*	Rem. Input	Action
S	xacc\$	Predict(1) S → ABc\$
ABc\$	xacc\$	Predict(2) A → xaA
xaABc\$	xacc\$	match(x)
aABc\$	acc\$	match(a)
ABc\$	cc\$	?

	x	y	a	b	c	\$
S	1	1			1	
A	2	3			4	
B				5	6	

Top-Down Parsing - Example

string: xacc\$

	x	y	a	b	c	\$
S	1	1			1	
A	2	3			4	
B				5	6	

Stack*

Rem. Input

Action

S	xacc\$	Predict(1) S->ABc\$
ABc\$	xacc\$	Predict(2) A->xaA
xaABc\$	xacc\$	match(x)
aABc\$	acc\$	match(a)
ABc\$	cc\$	Predict(4) A->c

Top-Down Parsing - Example

string: xacc\$

	x	y	a	b	c	\$
S	1	1			1	
A	2	3			4	
B				5	6	

Stack*

Rem. Input

Action

S	xacc\$	Predict(1) S->ABc\$
ABc\$	xacc\$	Predict(2) A->xaA
xaABc\$	xacc\$	match(x)
aABc\$	acc\$	match(a)
ABc\$	cc\$	Predict(4) A->c
cBc\$		

Top-Down Parsing - Example

string: xacc\$

	x	y	a	b	c	\$
S	1	1			1	
A	2	3			4	
B				5	6	

Stack*	Rem. Input	Action
S	xacc\$	Predict(1) S → ABc\$
ABc\$	xacc\$	Predict(2) A → xaA
xaABc\$	xacc\$	match(x)
aABc\$	acc\$	match(a)
ABc\$	cc\$	Predict(4) A → c
c Bc\$	c c\$	?

Top-Down Parsing - Example

string: xacc\$

	x	y	a	b	c	\$
S	1	1			1	
A	2	3			4	
B				5	6	

Stack*	Rem. Input	Action
S	xacc\$	Predict(1) S → ABc\$
ABc\$	xacc\$	Predict(2) A → xaA
xaABc\$	xacc\$	match(x)
aABc\$	acc\$	match(a)
ABc\$	cc\$	Predict(4) A → c
c Bc\$	cc \$	match(c)

Top-Down Parsing - Example

string: xacc\$

	x	y	a	b	c	\$
S	1	1			1	
A	2	3			4	
B				5	6	

Stack*	Rem. Input	Action
S	xacc\$	Predict(1) S → ABc\$
ABc\$	xacc\$	Predict(2) A → xaA
xaABc\$	xacc\$	match(x)
aABc\$	acc\$	match(a)
ABc\$	cc\$	Predict(4) A → c
cBc\$	cc\$	match(c)
B c\$	c \$	?

Top-Down Parsing - Example

string: xacc\$

	x	y	a	b	c	\$
S	1	1			1	
A	2	3			4	
B				5	6	

Stack*	Rem. Input	Action
S	xacc\$	Predict(1) S → ABc\$
ABc\$	xacc\$	Predict(2) A → xaA
xaABc\$	xacc\$	match(x)
aABc\$	acc\$	match(a)
ABc\$	cc\$	Predict(4) A → c
cBc\$	cc\$	match(c)
Bc\$	c\$	Predict(6) B → λ

Top-Down Parsing - Example

string: xacc\$

	x	y	a	b	c	\$
S	1	1			1	
A	2	3			4	
B				5	6	

Stack*	Rem. Input	Action
S	xacc\$	Predict(1) S → ABc\$
ABc\$	xacc\$	Predict(2) A → xaA
xaABc\$	xacc\$	match(x)
aABc\$	acc\$	match(a)
ABc\$	cc\$	Predict(4) A → c
cBc\$	cc\$	match(c)
B c\$	c\$	Predict(6) B → λ
c\$		

Top-Down Parsing - Example

string: xacc\$

	x	y	a	b	c	\$
S	1	1			1	
A	2	3			4	
B				5	6	

Stack*	Rem. Input	Action
S	xacc\$	Predict(1) S → ABc\$
ABc\$	xacc\$	Predict(2) A → xaA
xaABc\$	xacc\$	match(x)
aABc\$	acc\$	match(a)
ABc\$	cc\$	Predict(4) A → c
cBc\$	cc\$	match(c)
Bc\$	c\$	Predict(6) B → λ
c\$	c\$	?

Top-Down Parsing - Example

string: xacc\$

	x	y	a	b	c	\$
S	1	1			1	
A	2	3			4	
B				5	6	

Stack*	Rem. Input	Action
S	xacc\$	Predict(1) S → ABc\$
ABc\$	xacc\$	Predict(2) A → xaA
xaABc\$	xacc\$	match(x)
aABc\$	acc\$	match(a)
ABc\$	cc\$	Predict(4) A → c
cBc\$	cc\$	match(c)
Bc\$	c\$	Predict(6) B → λ
c\$	c\$	match(c)

Top-Down Parsing - Example

string: xacc\$

	x	y	a	b	c	\$
S	1	1			1	
A	2	3			4	
B				5	6	

Stack*	Rem. Input	Action
S	xacc\$	Predict(1) S → ABc\$
ABc\$	xacc\$	Predict(2) A → xaA
xaABc\$	xacc\$	match(x)
aABc\$	acc\$	match(a)
ABc\$	cc\$	Predict(4) A → c
cBc\$	cc\$	match(c)
Bc\$	c\$	Predict(6) B → λ
c\$	c\$	match(c)
\$	\$	Done!

Identifying LL(1) Grammar

- What we saw was an example of LL(1) Grammar
 - Scan input **L**eft-to-right, produce **L**eft-most derivation with **1** symbol look-ahead

Identifying LL(1) Grammar

- What we saw was an example of LL(1) Grammar
 - Scan input **L**eft-to-right, produce **L**eft-most derivation with 1 symbol look-ahead
- Not all Grammars are LL(1)

A Grammar is LL(1) iff for a production $A \rightarrow \alpha \mid \beta$, where α and β are distinct:

 1. For no terminal a do both α and β derive strings beginning with a (i.e. no common prefix)
 2. At most one of α and β can derive an empty string
 3. If $\beta \xRightarrow{*} \epsilon$, then α does not derive any string beginning with a terminal in $\text{Follow}(A)$. If $\alpha \xRightarrow{*} \epsilon$, then β does not derive any string beginning with a terminal in $\text{Follow}(A)$

Example (Left Factoring)

- Consider

`<stmt> → if <expr> then <stmt list> endif`

`<stmt> → if <expr> then <stmt list> else <stmt list> endif`

- This is not LL(1) (why?)
- We can turn this in to

`<stmt> → if <expr> then <stmt list> <if suffix>`

`<if suffix> → endif`

`<if suffix> → else <stmt list> endif`

Example (Left Factoring)

- Consider

`<stmt> → if <expr> then <stmt list> endif`

`<stmt> → if <expr> then <stmt list> else <stmt list> endif`

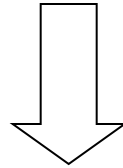
- This is not LL(1) (why?)
- We can turn this in to

`<stmt> → if <expr> then <stmt list> <if suffix>`

`<if suffix> → endif`

`<if suffix> → else <stmt list> endif`

Left Factoring

$$A \rightarrow \alpha \beta \mid \alpha \mu$$

$$A \rightarrow \alpha N$$
$$N \rightarrow \beta$$
$$N \rightarrow \mu$$

Left recursion

- *Left recursion* is a problem for LL(1) parsers
 - LHS is also the first symbol of the RHS
- Consider:
$$E \rightarrow E + T$$
- What would happen with the stack-based algorithm?

Left recursion

- *Left recursion* is a problem for LL(1) parsers
 - LHS is also the first symbol of the RHS

- Consider:

$$E \rightarrow E + T$$

- What would happen with the stack-based algorithm?

E

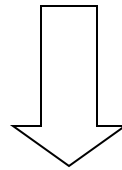
E + T

E + T + T

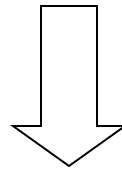
E + T + T + T

...

Eliminating Left Recursion

$$A \rightarrow A\alpha \mid \beta$$

$$A \rightarrow NT$$
$$N \rightarrow \beta$$
$$T \rightarrow \alpha T$$
$$T \rightarrow \lambda$$

Eliminating Left Recursion

$$E \rightarrow E + T \mid T$$

$$E \rightarrow E_1 \text{ Etail}$$
$$E_1 \rightarrow T$$
$$\text{Etail} \rightarrow + T \text{ Etail}$$
$$\text{Etail} \rightarrow \lambda$$

LL(k) parsers

- Can look ahead more than one symbol at a time
 - k -symbol lookahead requires extending first and follow sets
 - 2-symbol lookahead can distinguish between more rules:

$$A \rightarrow ax \mid ay$$

- More lookahead leads to more powerful parsers
- What are the downsides?

LL(k)? - Example

string: ((x+x))\$

- 1) $S \rightarrow E$
- 2) $E \rightarrow (E+E)$
- 3) $E \rightarrow (E-E)$
- 4) $E \rightarrow x$

Stack*	Rem. Input	Action
S	((x+x))\$	Predict(1) $S \rightarrow E$
E		Predict(2) or Predict(3)?

LL(1)

	(	+ -	)	x
S	1			1
E	2,3			4

LL(2)

	((	+(	-(	)\$	(x
S	1				1
E	2,3				4

Are all grammars LL(k)?

- No! Consider the following grammar:

$$\begin{aligned} S &\rightarrow E \\ E &\rightarrow (E + E) \\ E &\rightarrow (E - E) \\ E &\rightarrow x \end{aligned}$$

- When parsing E, how do we know whether to use rule 2 or 3?
- Potentially unbounded number of characters before the distinguishing '+' or '-' is found
- No amount of lookahead will help!

In real languages?

- Consider the if-then-else problem
- `if x then y else z`
- Problem: else is optional
- `if a then if b then c else d`
 - Which if does the else belong to?
- This is analogous to a “bracket language”: $[^i]^j$ ($i \geq j$)

$S \rightarrow [S C$

$S \rightarrow \lambda$

$C \rightarrow]$

$C \rightarrow \lambda$

$[[]$ can be parsed: $SS\lambda C$ or $SSC\lambda$
(it's ambiguous!)

Solving the if-then-else problem

- The ambiguity exists at the language level. To fix, we need to define the semantics properly
 - “[” matches nearest unmatched “[”
 - This is the rule C uses for if-then-else
 - What if we try this?

$$\begin{aligned} S &\rightarrow [S \\ S &\rightarrow SI \\ SI &\rightarrow [SI] \\ SI &\rightarrow \lambda \end{aligned}$$

This grammar is still not LL(1)
(or LL(k) for any k!)

Two possible fixes

- If there is an ambiguity, prioritize one production over another
- e.g., if C is on the stack, always match “]” before matching “ λ ”

$$\begin{array}{lcl} S & \rightarrow & [S C \\ S & \rightarrow & \lambda \\ C & \rightarrow &] \\ C & \rightarrow & \lambda \end{array}$$

- Another option: change the language!
- e.g., all if-statements need to be closed with an endif

$$\begin{array}{lcl} S & \rightarrow & \text{if } S \text{ E} \\ S & \rightarrow & \text{other} \\ E & \rightarrow & \text{else } S \text{ endif} \\ E & \rightarrow & \text{endif} \end{array}$$

Parsing if-then-else

- What if we don't want to change the language?
 - C does not require { } to delimit single-statement blocks
- To parse if-then-else, we *need to be able to look ahead at the entire rhs of a production* before deciding which production to use
 - In other words, we need to determine how many “]” to match before we start matching “[”s
- *LR parsers* can do this!

Hand-Written Parser - FPE

- Fully parenthesized expression (FPE)
 - Expressions (algebraic notation) are the normal way we are used to seeing them. E.g. $2 + 3$
 - *Fully-parenthesized* expressions are simpler versions: every binary operation is enclosed in parenthesis
 - E.g. $2 + 3$ is written as $(2+3)$
 - E.g. $(2 + (3 * 7))$
 - We can ignore order-of-operations (PEMDAS rule) in FPEs.

FPE – definition

- Either a:
 1. A number (integer in our example) OR
 2. *Open parenthesis* ‘(’ followed by
FPE followed by
an operator (‘+’, ‘-’, ‘*’, ‘/’) followed by
FPE followed by
closed parenthesis ‘)’

FPE – Notation

1. $E \rightarrow \text{INTLITERAL}$

2. $E \rightarrow (E \text{ op } E)$

3. $\text{op} \rightarrow \text{ADD} \mid \text{SUB} \mid \text{MUL} \mid \text{DIV}$

Implementing a parser for FPE

1. One function defined for every non-terminal

- E, op

2. One function defined for every production

- E1, E2

3. One function defined for all terminals

- IsTerm

1.E -> INTLITERAL

2.E -> (E op E)

3.op -> ADD | SUB | MUL | DIV

Implementing a parser for FPE

1. One function defined for every non-terminal

- E, op

2. One function defined for every production

- E1, E2

3. One function defined for all terminals

- IsTerm

1.E $\rightarrow$ INTLITERAL

2.E $\rightarrow$ (E op E)

3.op $\rightarrow$ ADD | SUB | MUL | DIV

Implementing a parser for FPE

This function checks if the next token returned by the scanner matches the expected token. Returns true if match. false if no match.

Assume that a scanner module has been provided.
The scanner has one function, `GetNextToken`, that
returns the next token in the sequence.

Can be any one of: `INTLITERAL`, `LPAREN`,
`RPAREN`, `ADD`, `SUB`, `MUL`, `DIV`



```
bool IsTerm(Scanner* s, TOKEN tok) {  
    return s->GetNextToken() == tok;  
}
```

Implementing a parser for FPE

1. One function defined for every non-terminal

- E, op

2. One function defined for every production

- E1, E2

3. One function defined for all terminals

- IsTerm

1.E $\rightarrow$ INTLITERAL

2.E $\rightarrow$ (E op E)

3.op $\rightarrow$ ADD | SUB | MUL | DIV

Implementing a parser for FPE

This function implements production #1: $E \rightarrow \text{INTLITERAL}$

Returns true if the next token returned by the scanner is an INTLITERAL. false otherwise.

```
bool E1(Scanner* s) {  
    return IsTerm(s, INTLITERAL);  
}
```

Implementing a parser for FPE

1. One function defined for every non-terminal

- E, op

2. One function defined for every production

- E1, E2

3. One function defined for all terminals

- IsTerm

1.E $\rightarrow$ INTLITERAL

2.E $\rightarrow$ (E op E)

3.op $\rightarrow$ ADD | SUB | MUL | DIV

Implementing a parser for FPE

This function implements production #2: $E \rightarrow (E \text{ op } E)$

Returns true if the Boolean expression on line 2 returns true. false otherwise.

```
1: bool E2(Scanner* s) {  
  
2:   return IsTerm(s, LPAREN) &&  
        E(s) &&  
        OP(s) &&  
        E(s) &&  
        IsTerm(s, RPAREN);  
  
3: }
```

Implementing a parser for FPE

1. One function defined for every non-terminal

- E, op

2. One function defined for every production

- E1, E2

3. One function defined for all terminals

- IsTerm

1.E $\rightarrow$ INTLITERAL

2.E $\rightarrow$ (E op E)

3.op $\rightarrow$ ADD | SUB | MUL | DIV

Implementing a parser for FPE

This function implements production #3: $op \rightarrow \text{ADD} \mid \text{SUB} \mid \text{MUL} \mid \text{DIV}$

Returns true if the next token returned by the scanner is any one from ADD, SUB, MUL, DIV. false otherwise.

```
bool OP(Scanner* s) {  
  
    TOKEN tok = s->GetNextToken();  
  
    if((tok == ADD) || (tok == SUB) || (tok ==  
        MUL) || (tok == DIV))  
        return true;  
  
    return false;  
  
}
```


Implementing a parser for FPE

1. One function defined for every non-terminal

- **E**, op

2. One function defined for every production

- E1, E2

3. One function defined for all terminals

- IsTerm

1.E $\rightarrow$ INTLITERAL

2.E $\rightarrow$ (E op E)

3.op $\rightarrow$ ADD | SUB | MUL | DIV

Implementing a parser for FPE

This function implements the routine for matching non-terminal E

```
bool E(Scanner* s) {  
    TOKEN* prevToken = s->GetCurTokenSequence();  
    if(!E1(s)) {  
        s->SetCurTokenSequence(prevToken);  
        return E2(s);  
    }  
    return true;  
}
```

Assume that GetCurTokenSequence
returns a reference to the first token in
a sequence of tokens maintained by
the scanner



Implementing a parser for FPE

This function implements the routine for matching non-terminal E

```
bool E(Scanner* s) {  
  
    TOKEN* prevToken = s->GetCurTokenSequence();  
    if(!E1(s)) {  
        s->SetCurTokenSequence(prevToken);  
        return E2(s);  
    }  
    return true;  
}
```

//This line implements the check to see if the sequence of tokens match production #1:
E->INTLITERAL.

Implementing a parser for FPE

This function implements the routine for matching non-terminal E

```
bool E(Scanner* s) {  
  
    TOKEN* prevToken = s->GetCurTokenSequence();  
    if(!E1(s)) {  
        s->SetCurTokenSequence(prevToken);  
        return E2(s);  
    }  
    return true;  
}
```

//because E1(s) calls s->GetNextToken() internally, the reference to the sequence of tokens would have moved forward. This line restores the reference back to the first node in the sequence so that the scanner provides the correct sequence to the call E2 in next line

Implementing a parser for FPE

This function implements the routine for matching non-terminal E

```
bool E(Scanner* s) {  
  
    TOKEN* prevToken = s->GetCurTokenSequence();  
    if(!E1(s)) {  
        s->SetCurTokenSequence(prevToken);  
        return E2(s);  
    }  
    return true;  
}
```

```
//This line implements the check to see if the sequence of tokens match production #2:  
E->(E op E)
```

Implementing a parser for FPE

```
IsTerm(Scanner* s, TOKEN tok) { return s->GetNextToken() == tok;}

bool E1(Scanner* s) {
    return IsTerm(s, INTLITERAL);
}

bool E2(Scanner* s) { return IsTerm(s, LPAREN) && E(s) && OP(s) && E(s) && IsTerm(s, RPAREN); }

bool OP(Scanner* s) {
    TOKEN tok = s->GetNextToken();
    if((tok == ADD) || (tok == SUB) || (tok == MUL) || (tok == DIV))
        return true;
    return false;
}

bool E(Scanner* s) {
    TOKEN* prevToken = s->GetCurTokenSequence();
    if(!E1(s)) {
        s->SetCurTokenSequence(prevToken);
        return E2(s);
    }
    return true;
}
```

Start the parser by invoking E().
Value returned tells if the expression is FPE or not.

Exercise

- What parsing technique does this parser use?

Bottom-up Parsing

- More general than top-down parsing
- Used in most parser-generator tools
- Need not have left-factored grammars (i.e. can have left recursion)
- E.g. can work with the bracket language

Bottom-up Parsing

- Reduce a string to start symbol by reverse 'inverting' productions

Bottom-up Parsing

- Reduce a string to start symbol by reverse 'inverting' productions

id * id + id

$E \rightarrow T + E$

$E \rightarrow T$

$T \rightarrow id * T$

$T \rightarrow id$

Bottom-up Parsing

- Reduce a string to start symbol by reverse 'inverting' productions

id * id + id
id * T + id

E $\rightarrow$ T + E
E $\rightarrow$ T
T $\rightarrow$ id * T
T $\rightarrow$ id

Bottom-up Parsing

- Reduce a string to start symbol by reverse 'inverting' productions

id * id + id
id * T + id
T + id

E $\rightarrow$ T + E

E $\rightarrow$ T

T $\rightarrow$ id * T

T $\rightarrow$ id

Bottom-up Parsing

- Reduce a string to start symbol by reverse 'inverting' productions

id * id + id
id * T + id
T + id
T + T

E $\rightarrow$ T + E
E $\rightarrow$ T
T $\rightarrow$ id * T
T $\rightarrow$ id

Bottom-up Parsing

- Reduce a string to start symbol by reverse 'inverting' productions

id * id + id
id * T + id
T + id
T + T
T + E

E $\rightarrow$ T + E

E $\rightarrow$ T

T $\rightarrow$ id * T

T $\rightarrow$ id

Bottom-up Parsing

- Reduce a string to start symbol by reverse 'inverting' productions

id * id + id

id * T + id

T + id

T + T

T + E

E

E $\rightarrow$ T + E

E $\rightarrow$ T

T $\rightarrow$ id * T

T $\rightarrow$ id

Bottom-up Parsing

- Reduce a string to start symbol by reverse 'inverting' productions

id * id + id

id * T + id

T + id

T + T

T + E

E



E $\rightarrow$ T + E

E $\rightarrow$ T

T $\rightarrow$ id * T

T $\rightarrow$ id

Bottom-up Parsing

- Reduce a string to start symbol by reverse 'inverting' productions

id * id + id
id * T + id
T + id
T + T
T + E
E

Right-most derivation

$E \rightarrow T + E$
 $E \rightarrow T$
 $T \rightarrow id * T$
 $T \rightarrow id$

Bottom-up Parsing

- Scan the input left-to-right and **shift** tokens – put them on the stack.

| id * id + id

id | * id + id

id * | id + id

id * id | + id

$E \rightarrow T + E$

$E \rightarrow T$

$T \rightarrow id * T$

$T \rightarrow id$

Bottom-up Parsing

- Replace a set of symbols at the top of the stack that are RHS of a production. Put the LHS of the production on stack – **Reduce**

| id * id + id

id | * id + id

id * | id + id

id * id | + id

$E \rightarrow T + E$

$E \rightarrow T$

$T \rightarrow id * T$

$T \rightarrow id$

Bottom-up Parsing

- Did not discuss when and why a particular production was chosen

id * id + id
id * T + id

E $\rightarrow$ T + E
E $\rightarrow$ T
T $\rightarrow$ id * T
T $\rightarrow$ id

- i.e. why replace the id highlighted in input string?*

LR Parsers

- Parser which does a Left-to-right, Right-most derivation
 - Rather than parse top-down, like LL parsers do, parse bottom-up, starting from leaves
- Basic idea: put tokens on a stack until an entire production is found
- Issues:
 - Recognizing the endpoint of a production
 - Finding the length of a production (RHS)
 - Finding the corresponding nonterminal (the LHS of the production)

Data structures

- At each state, given the next token,
 - A *goto table* defines the successor state
 - An *action table* defines whether to
 - *shift* – put the next state and token on the stack
 - *reduce* – an RHS is found; process the production
 - *terminate* – parsing is complete